AI PRODUCT MANAGEMENT 2 · DETAILED COURSE

Technical Foundations for Product Managers

Enough software and systems knowledge to discuss trade-offs credibly with engineers and make sound decisions.

Chapter 10 of 26 · 28 topics · Edition 2026 · skillnavora.in

CHAPTER 10

Technical Foundations for Product Managers

Enough software and systems knowledge to discuss trade-offs credibly with engineers and make sound decisions.

How Software Applications Work

Users interact with a front end that calls back-end services, which read and write data and return results.

A user interacts with a front end, which sends requests to back-end services. Those services apply business rules, read and write data and return results. Knowing this flow helps a PM describe where a bug or delay happens.

Software Development Lifecycle (SDLC)

Plan, design, build, test, deploy and maintain, in sequential or iterative form.

The SDLC covers planning, design, build, test, deploy and maintain. It can run in sequence or iteratively. Each stage has outputs such as requirements, designs, code and release notes that the PM reviews.

Frontend Fundamentals

HTML, CSS and JavaScript in the browser or native code on mobile, handling what users see and touch.

Frontend code (HTML, CSS and JavaScript on the web, or native code on mobile) controls what users see and touch. Performance on frontend depends on page weight, rendering and network requests.

Backend Fundamentals

Server logic, business rules, databases and integrations that power the front end.

The back end runs server logic, business rules, databases and integrations. Latency and reliability problems often start here, so PMs should understand what runs on the back end and what each request costs.

Client-Server Architecture

Clients send requests; servers process and respond, over a network.

Clients send requests and servers process them and send responses over a network. Most apps follow this pattern. Consider where data is processed when designing offline or low-bandwidth features.

Databases and Data Storage

Persistent stores for application data, chosen for structure, scale and query needs.

Databases persist application data. The choice depends on structure, scale, consistency and query needs. Changing a database later is expensive, so involve engineers early in data decisions.

SQL and NoSQL Databases

SQL stores structured tables with strong consistency; NoSQL covers document, key-value, column and graph stores built for flexibility and scale.

SQL databases store structured tables with strong consistency and joins. NoSQL covers document, key-value, column and graph stores that trade some consistency or queryability for flexibility and scale.

Relational Database Concepts

Tables, keys, relationships, normalisation and transactions (ACID).

Relational databases use tables, keys, relationships and normalisation. Transactions follow ACID properties (atomicity, consistency, isolation, durability), which keep money movements correct.

APIs and API Fundamentals

Contracts that let software components talk, with defined requests, responses and rules.

An API is a contract that lets software components communicate. It defines requests, responses, authentication and error behaviour. Good APIs are stable and documented, because other teams depend on them.

REST APIs

Resource-based HTTP APIs using verbs like GET, POST, PUT and DELETE and status codes.

REST exposes resources through HTTP verbs such as GET, POST, PUT and DELETE, and uses status codes to report results. Example: GET /orders/123 returns one order.

GraphQL

A query language where clients request exactly the fields they need from one endpoint.

GraphQL lets clients request exactly the fields they need from a single endpoint. It reduces over-fetching for complex screens but needs care with query cost and caching.

Webhooks

Event-driven callbacks where one system sends an HTTP request to another when something happens.

A webhook sends an HTTP request from one system to another when an event occurs, such as payment succeeded. It avoids constant polling, but the receiver must handle retries and duplicate events.

Authentication and Authorisation

Authentication proves who you are; authorisation decides what you may do (OAuth, SSO, roles).

Authentication proves identity, for example with a password or single sign-on. Authorisation decides what that identity may do, using roles or permissions. OAuth is a common standard for delegated access.

JSON and XML

Text formats for exchanging structured data; JSON is lighter and more common today.

JSON and XML are text formats for structured data. JSON is lighter and standard for web APIs. Knowing the format helps a PM read API responses and specifications.

API Documentation

Reference, guides and examples (often OpenAPI) that make APIs easy to adopt.

Good documentation includes reference pages, guides and examples, often generated from an OpenAPI specification. Clear docs cut support requests and speed up integration.

Postman

A tool to explore, test and document APIs by sending requests and viewing responses.

Postman lets you send requests, inspect responses and save collections of API calls. PMs use it to understand how an API behaves before writing requirements.

Git and GitHub Fundamentals

Git tracks changes; GitHub hosts repositories and supports pull requests and reviews.

Git tracks changes to code. GitHub hosts repositories and supports pull requests and code review. Knowing the basic workflow lets a PM follow release discussions and documentation changes.

Version Control

Recording history of changes so teams can collaborate, branch, review and roll back.

Version control records history so teams can collaborate, branch, review and roll back. Rollbacks depend on it, and it is the audit trail for what changed in each release.

Microservices Architecture

Many small, independently deployable services that communicate over APIs.

Microservices split an application into small, independently deployable services that talk over APIs. They help large teams work in parallel, but add operational complexity and network failure modes.

Monolithic Architecture

One deployable unit containing all functionality: simple to start, harder to scale teams.

A monolith is one deployable unit containing all functionality. It is simpler to start and debug, but large teams can slow each other down. Many successful products begin as monoliths.

System Design Fundamentals

Choosing components, data flow and trade-offs to meet requirements for scale, cost and reliability.

System design chooses components, data flows and trade-offs to meet requirements for scale, cost and reliability. PMs do not design systems alone, but they should know which trade-offs their requirements create.

Scalability and Reliability

Handling growth in load and staying available despite failures, using redundancy, caching and graceful degradation.

Scalability handles growing load, and reliability keeps the service available despite failures. Techniques include redundancy, caching, load balancing and graceful degradation, such as showing cached data when a service is slow.

Latency and Throughput

Latency is time per request; throughput is requests handled per unit time.

Latency is the time for one request. Throughput is the number of requests handled per unit time. A system can have high throughput and poor latency, so track both.

Cloud Computing Fundamentals

On-demand computing, storage and services paid by use: IaaS, PaaS, SaaS, regions and shared responsibility.

Cloud services provide compute, storage and managed services on demand, paid by use. Models include IaaS, PaaS and SaaS. Responsibility for security is shared between provider and customer.

AWS Fundamentals

Amazon's cloud: EC2, S3, Lambda, RDS and IAM as core building blocks.

AWS offers services such as EC2 for virtual machines, S3 for storage, Lambda for functions, RDS for managed databases and IAM for access control. Knowing these names helps in technical discussions.

Microsoft Azure Fundamentals

Microsoft's cloud: virtual machines, storage, Azure AD, Azure OpenAI and DevOps.

Azure provides virtual machines, storage, Azure Active Directory for identity, Azure OpenAI for model access and DevOps tools. Many enterprises already use Azure, which affects procurement and integration choices.

Google Cloud Platform Fundamentals

Google's cloud: Compute Engine, Cloud Storage, BigQuery and Vertex AI.

GCP offers Compute Engine, Cloud Storage, BigQuery for analytics and Vertex AI for machine learning. Its data and AI services are often chosen for analytics-heavy products.

Cybersecurity Fundamentals

Confidentiality, integrity, availability, encryption, secure coding, access control and common attacks.

Security covers confidentiality, integrity and availability, plus encryption, secure coding, access control and common attacks. A PM must treat security requirements as product requirements, not as later clean-up.

Summary of terms

Term	Meaning
How Software Applications Work	Users interact with a front end that calls back-end services, which read and write data and return results.
Software Development Lifecycle (SDLC)	Plan, design, build, test, deploy and maintain, in sequential or iterative form.
Frontend Fundamentals	HTML, CSS and JavaScript in the browser or native code on mobile, handling what users see and touch.
Backend Fundamentals	Server logic, business rules, databases and integrations that power the front end.
Client-Server Architecture	Clients send requests; servers process and respond, over a network.
Databases and Data Storage	Persistent stores for application data, chosen for structure, scale and query needs.
SQL and NoSQL Databases	SQL stores structured tables with strong consistency; NoSQL covers document, key-value, column and graph stores built for flexibility and scale.
Relational Database Concepts	Tables, keys, relationships, normalisation and transactions (ACID).
APIs and API Fundamentals	Contracts that let software components talk, with defined requests, responses and rules.
REST APIs	Resource-based HTTP APIs using verbs like GET, POST, PUT and DELETE and status codes.
GraphQL	A query language where clients request exactly the fields they need from one endpoint.
Webhooks	Event-driven callbacks where one system sends an HTTP request to another when something happens.
Authentication and Authorisation	Authentication proves who you are; authorisation decides what you may do (OAuth, SSO, roles).
JSON and XML	Text formats for exchanging structured data; JSON is lighter and more common today.
API Documentation	Reference, guides and examples (often OpenAPI) that make APIs easy to adopt.
Postman	A tool to explore, test and document APIs by sending requests and viewing responses.
Git and GitHub Fundamentals	Git tracks changes; GitHub hosts repositories and supports pull requests and reviews.
Version Control	Recording history of changes so teams can collaborate, branch, review and roll back.
Microservices Architecture	Many small, independently deployable services that communicate over APIs.
Monolithic Architecture	One deployable unit containing all functionality: simple to start, harder to scale teams.
System Design Fundamentals	Choosing components, data flow and trade-offs to meet requirements for scale, cost and reliability.
Scalability and Reliability	Handling growth in load and staying available despite failures, using redundancy, caching and graceful degradation.

Latency and Throughput	Latency is time per request; throughput is requests handled per unit time.
Cloud Computing Fundamentals	On-demand computing, storage and services paid by use: IaaS, PaaS, SaaS, regions and shared responsibility.
AWS Fundamentals	Amazon's cloud: EC2, S3, Lambda, RDS and IAM as core building blocks.
Microsoft Azure Fundamentals	Microsoft's cloud: virtual machines, storage, Azure AD, Azure OpenAI and DevOps.
Google Cloud Platform Fundamentals	Google's cloud: Compute Engine, Cloud Storage, BigQuery and Vertex AI.
Cybersecurity Fundamentals	Confidentiality, integrity, availability, encryption, secure coding, access control and common attacks.

Practice exercise: Call a public REST API in Postman, read the JSON response, and draw the client, server and database flow for that request.

Self-check

1. Explain How Software Applications Work in two sentences to a non-technical stakeholder, then say what you would measure for it.
2. Explain API Documentation in two sentences to a non-technical stakeholder, then say what you would measure for it.
3. Explain Cybersecurity Fundamentals in two sentences to a non-technical stakeholder, then say what you would measure for it.

Related SkillNavora material

- System Design and Databases for PMs: Architecture, databases and trade-offs.
- Platform and API Product Management: API products, versioning and developer experience.
- AI Tutor: Ask questions on any topic.
- Study Library: Every lesson, class and PDF by programme.

© SkillNavora · skillnavora.in · Facts and tools change; check official sites before relying on a figure.